Document number: D2339R0
Date: 2021-03-15
Audience: SG21
Reply-to: *Andrzej Krzemieński <akrzemi1 at gmail dot com>*

Contract violation handlers

This document tries to put some structure to the recent reflector discussion about what a contract violation handler should be allowed to do. There are basically three possibilities:

1. It ends in calling `std::terminate()` or `std::abort()`.
2. It ends by throwing an exception.
3. It returns *normally* (like a function that calls `return;`).

The first option seems uncontroversial. The other two options face objections. Throwing an exception introduces a new control flow only for the path that contains a bug, which is likely not tested and one that the program may not be prepared to handle. Returning normally does not protect against the likely subsequent undefined behavior, which may result even in time travel and skipping the entire runtime check.

This document tries to describe in detail the use cases for non-aborting violation handlers. The hope is to build the common understanding of these use cases and their consequences within SG12.

1. Discussion

1.1. Expressing preconditions in code

Preconditions in general are something that does not belong to code. They are part of software documentation: part of human-to-human communication. This is an example:

```
int server_perform(Task * task);
  // Prerequisites:
  // * `task` points to a valid object of type Task or derived
  // * `task` has not already been performed
  // * function `server_init` has been called prior to calling this function
```

Only a subset of these prerequisites can be expressed in code. Using the notation from C++20 contracts ([P0542r5]), one could express a subset of these prerequisites like this:

```
int server_perform(Task * task)
  [[pre: task != nullptr]]
  [[pre: !task->performed()]];
  // Prerequisites:
  // * `task` points to a valid object of type Task or derived
  // * function `server_init` has been called prior to calling this function
```

However, if it is a policy in a given project that pointers passed and returned from functions are never null unless explicitly indicated, then typing `p != nullptr` on every function may only add noice to the interface and distract human attention from parts specific to a documented function. In order to address this, one could use C++ contracts in a different way: put contract annotations inside the function body:

```
// header file:

int server_perform(Task * task);
  // Prerequisites:
  // * `task` points to a valid object of type Task or derived
  // * `task` has not already been performed
  // * function `server_init` has been called prior to calling this function

// source file:

int server_perform(Task * task)
{
  [[assert: task != nullptr]];
  [[assert: !task->performed()]];

  // function logic ....
}
```

In fact, testing parts of the precondition inside function body is what most libraries for contract-checking do today. For instance, C++ Core Guidelines ([CppCoreGuidelines]) recommend doing this. Admittedly, current libraries often do this, because there is no way to put these annotations into function declarations. But even if this possibility is added to the language, there may be cases where in-body annotations would be preferred:

```
template<forward_range R, class T,
         indirect_strict_weak_order<T*, iterator_t<R>> Comp = ranges::less>
  constexpr bool binary_search(R&& r, const T& value, Comp comp = {})
  {
    auto checked_pred = [&](iterator_t<R>> lhs, iterator_t<R>> rhs)
    {
      if (comp(lhs, rhs))
      {
        [[assert: !comp(rhs, lhs)]]; // the subset of requirements in `indirect_strict_weak_order`
        return true;
      }

      return false;
    };

    _binary_search(std::move(r), value, checked_pred);
  }
```

The runtime complexity of `binary_search` is O(log(*n*)). If we wanted to check the correctness of the predicate for all input values, if it respects the strict weak ordering rules, we would increase the complexity to O(n^2). We could turn it into an `audit`-level check, but it would never be usable in production. But if we test the predicate only on the values that we actually visit, we get back the complexity of O(log(*n*)). We could extract this hack into a dedicated function, but the way it is expressed in the above example feels simpler.

1.2. What do preconditions do?

What effect do precondition declarations have on the compiled binaries? For each precondition declaration there are two possible outcomes:

- either the declaration does not affect the binary,
- or an additional runtime code, possibly with side effects is injected.

Which of the two it is, is dependent on factors external to the source code, such as build modes. What code is actually injected is still an open issue and the subject of this paper. The answer depends on whether we allow the handlers to throw or return normally.

Given the following declaration,

```
int f(int* p)
  [[pre: p != nullptr]]
{
  return *p;
}
```

Assuming that the user-installed violation handler is accessible through function `violation_handler()`, the code injected for a terminating violation handler could look like the following:

```
int f(int* p)
{
  if ((p != nullptr) == false)
  {
    []() noexcept { violation_handler(); }(); // immediately invoked lambda
    std::terminate();
  }

  return *p;
}
```

The call to the handler is wrapped into a `noexcept` lambda so that any exception trown from inside the handler is turned into a call to `std::terminate()`. A different implementation of terminating handler would be to statically enforce that the registered handler is declared as `noexcept`:

```
int f(int* p)
{
  if ((p != nullptr) == false)
  {
    static_assert(noexcept(violation_handler()));
    violation_handler();
    std::terminate();
  }

  return *p;
}
```

If throwing violation handlers are allowed, the "runtime check" generated from the precondition would be equivalent to:

```
int f(int* p)
{
  if ((p != nullptr) == false)
  {
    violation_handler();
    std::terminate();
  }

  return *p;
}
```

Finally, a use case where the failed runtime check does not necessarily stop the execution of the function could be handled by the following generated runtime code:

```
int f(int* p)
{
  if ((p != nullptr) == false)
  {
    violation_handler();
  }

  return *p;
}
```

1.3. What are preconditions used for?

Let's consider three use cases. First, a precondition expresses a condition which unsatisfied leads to undefined behavior. We have seen an example of this in the previous section:

```
int f(int* p)
  [[pre: p != nullptr]]
{
  return *p; // UB if this pointer is null
}
```

Second use case is when we express a condition which when violated has a well defined behavior in C++, but its consequences are known to be grave. Consider a precondition that expresses the potential SQL injection:

```
int Server::user_count(std::string user_name)
  [[pre: is_sanitized(user_name)]]
{
  return execute(build_sql(user_name));
}
```

Third use case is when upon a violated precondition nothing inside our function can "break", but it is clear that there is a bug in the caller. We provide three examples for this use case.

```
int f(int* p)
  [[pre: p != nullptr]]
{
  if (p == nullptr)
    throw std::logic_error("bad pointer value");

  return *p; // never null
}
```

Here, the function protects itself against the incorrect usage, and signals it at runtime. But we still declare it in the precondition in order to signal that such situation is a bug, and help tools like IDEs or static analyzers detect this bug statically rather than waiting till run time.

```
bool is_in_range(int val, int min, int max)
  [[pre: min <= max]]
{
  return min <= val && val <= max;
}
```

In the above example we are comfortable with returning *some* result for any combination of input values. However, we are able to tell that if `min > max` there must be a bug in the caller, because then `min` and `max` no longer represent a range, so the function call is meaningless.

The last illustration:

```
Cache::Cache(size_t size)
  [[size > 0]]
  : vector(size)
{
  expensive_initialization_of_auxiliaty_data_structures();
}
```

Creating a cache of size zero is technically correct. The application will perform correctly, but inefficiently. There is no point in creating a zero-sized cache as you pay the cost of maintaining the data structure, but you are actually caching nothing. If this happens, it is possible that a user misunderstood the interface of our cache. By using the precondition, we enable any tool that makes use of it to alert the user that their solution can be improved by using the interface correctly.

All the above use case are quite different, but they have one thing in common: in each case we detect that the program has a bug: big or small, fatal or benign, but definately a bug.

1.4. Programming models for contract annotations and contract violations

There are at least two ways of looking at what contract annotations represent:

1. They declare what constitutes a bug in the program.
2. They are a control flow mechanism, customizable through compiler switches.

While to some extend these two views can co-exist, they have also incompatible differences. For instance, under the first model it makes sense to allow the compiler to refuse to compile the code and report an error when it can prove that the program when run would inevitably cause a contract expressed by an annotation to be violated.

Under the second model, when the programmer knows that the violation handler ends in throwing an exception, she can use contract annotations like this:

```
void throw_if_contract_checking_enabled()
  // Sys owner: remember to always install a throwing handler
  [[pre: false]]
{}

int main()
{
  try {
    throw_if_contract_checking_enabled();
    std::cout << "Hello, contract checking is disabled \n";
  }
  catch(...) {
    std::cout << "Hello, contract checking is enabled \n";
  }

}
```

1.5. Use cases for violation handlers?

1.5.1. Prevent damage

The most obvious and uncontroversial is the use case for preventing any damage that a program with runtime-confirmed bug may cause. We suspect that our program contains bugs and some of them may have grave consequences, like allowing unauthorized access or returning random results. We want to minimize the risk, so whenever our declared preconditions (or other contract annotations) detect one of these potential situations, we want to shut down the application immediately.

1.5.2. Inform about runtime-detected potentially critical checks

Just to kill the application may not be satisfactory. We may also expect that the application will communicate somehow that it made a deliberate decision to stop, and can give us some context that would help us find the bug. This can be a short message displayed to standard output, or a core dump. This part wil be covered by the violation handler. But in order for this to work, we need the program to terminate. It is only the halting of the program that gives us the guarantee that:

- The program will not execute beyond the failed contract point.
- The handler will be called for a failed contract check, regardless of what source code follows the precondition declaration.

1.5.3. Test the existence of defensive checks

If precondition annotations are put in function declarations, it is easy to visually confirm that they are there. However, if the library is interested in only performing runtime defensive checks inside the body of the function as in:

```
int server_perform(Task * task)
{
  [[assert: task != nullptr]];
  [[assert: !task->performed()]];

  // function logic ....
}
```

one might want to unit-test if these defensive checks have been put in place. The way to do it is to actually call the function out of contract and see if the violation handler has been triggered. There are two tricky parts about this process, though.

1. This is the only place where we deliberately call a function with its contract violated. This breaks the programming model which treats each contract violation as a bug that requires fixing. Implementations have license to diagnose it and even should be allowed to refuse to compile such code. This may confuse the tools (static analyzers, IDEs). Of course, this is just a proposed programming model. Under a different model, contract annotations are just control statements (like `throw`). Unit-testing of contract violations relies on this weaker model.
2. The function that called the handler must not be allowed to continue (it may be not prepared for such case), but at the same time we need the test program to continue in order to build the test report.

While the first problem remains unresolved, there are two known ways of solving the second one. One is to use "death tests", as called in [GTest] (see https://github.com/google/googletest/blob/master/docs/advanced.md#death-tests). In short, for each such test we spawn a new process and in that process call the function with violated contract; then we check if the process terminated. This process does not work in all cases, though; and the time it takes to exectue the "negative" tests this way is noticeable.

The other way of testing the existence of defensive runtime-checks is to install, for the purpose of performing unit tests, a contract violation handler that throws a unique exception type representing the detected violation. Now the test can check if the expected exception is thrown when bad function arguments are passed.

There are two objections to throwing violation handlers.

1. A program that caused the contract violation is allowed to continue.
2. Functions tested this way cannot be declared `noexcept`.

Regarding the first objection. While a throw skips most of the function body, some subsequent instructions inside the function are still executed: destructors. Destructors also require that the preceeding code has not violated the contract. Destructors can also have preconditions that need to be satisfied. Second, while the body of the function may be skipped, the calling code will likely catch the exception at some point, and continue the normal execution. This requires weakening the programming model and treating contract violations as an ordinary control flow mechanism.

This part of the criticism assumes that throwing handlers are used in programs that potentially run in production environments. But this is not the case. Users who wish to test the presence of defensive checks in this way will only install throwing violation handlers in their unit-test programs: never in the shipped executables. The unit-test environment is special here: (1) only the developers are consumers, (2) exceptions from violation handlers never travel long through the call stack: they are immediately caught at the next call stack frame.

Regarding the second objection. This actually requires to answer a different question first: what is `noexcept` for?

1. Is it only for move constructor and move assignment, as was the original intent, reflected in [N3050]?
2. Is it a guarantee that the function will never throw, so that compilers can skip generating the stack unwinding code?

If the answer is 1, then the "`noexcept`" objection to throwing violation handlers is moot. Move operations practically always have wide contracts. If the answer is 2, then the objection becomes real. Library vendors who chose to test their libraries this way would have to make sure that their functions with narrow contracts are never `noexcept`, this will affect their consumers, who will get libraries that generate suboptimal executables (in terms of performance).

1.5.4. Check for benign bugs

A programmer may know that some contract annotations represent benign bugs. Sometimes this is because the programmer knows the properties of a given component, like the implementation of cache, where zero size will work, but suboptimally. Sometimes the programmer can assume that certain bugs are benign through the following process. If a piece of software has been running in production for a couple of years and has stabilized, it is safe to assume that it has no serious bugs. In the next release, one can add contract annotations that were previously absent. If they are now runtime-checked and report the violation, it is safe to assume that thus discovered bugs are benign. At this point terminating the program would be a bad decision. So, we need a handler that will only log the violation event, and let the program continue.

There are two objections to violation handlers that just return normally (and let the program continue). First, we loose the guarantee that doesn't allow the program to execute beyond the failed runtime check.

Second. some preconditions protect against the undefined behavior inside the function body. If the execution continues after the failed runtime check, such undefined behavior is engaged, and its consequences can travel back through time and cause the preceding runtime check to be erased. As a result, the programmer thinks that the condition is runtime-checked because runtime checking has been enabled by a compiler switch, but the precondition check is in fact skipped due to the subsequent undefined behavior. The following example illustrates this:

```
namespace
{
  int bug_count = 0;
```

```
  void violation_handler() { ++bug_count; }
}

int fun(int * p)
  [[pre: p != nullptr]]
{
  return *p;  // UB if p is null
}

int gun(int* p)
{
  return fun(p) + bug_count;
}
```

When runtime checks are generated from preconditions, and the program is allowed to continue after calling the violaiton handler, the generated code will be equivalent to:

```
namespace
{
  int bug_count = 0;
  void violation_handler() { ++bug_count; }
}

int fun(int * p)
{
  return *p;  // UB if p is null
}

int gun(int* p)
{
  if (!p)                // injected from precondition
    violation_handler(); //

  return fun(p) + bug_count;
}
```

One might expect that the injected check is performed and the handler executed, but an optimizing compiler can observe that if `p` passed to `gun` is null, then it will be passed to `fun` and there cause undefined behavior (UB). The compiler is allowed to assume that UB never happens; from this it can conclude that `p` is never null, and based on this conclusion it can eliminate the defensive check in `gun`. Compilers really do this today, as demonstrated in this Compiler Explorer example: https://godbolt.org/z/Yz73z4.

In response to this concern, we could say that continuing violation handlers are a sharp tool that should be used only under a strict regime. This regime is: first test your application thoroughly for any bugs or UB, and if nothing shows up only then add new preconditions that allow continuation. If any precondition violation is runtime-detected at that point, it must be a benign bug, so runtime check cannot be magically compromised by the subsequent UB.

However, consider a different example, containing a benign bug:

```
int f(int * p)
  [[pre: p]]
  [[pre: *p > 0]]
{
  if (!p || *p <= 0)      // safety double-check
    return std::logic_arror("");

  return *p - 1;
}
```

If this function is invoked in a program that doesn't runtime-check contract annotations, it behaves in a tolerable way: it throws an exception. But when runtime checking is enabled and the violation handler returns normally, this code is equivalent to:

```
int f(int * p)
{
  if (!p)                 // (1)
    violation_handler();

  if (*p <= 0)            // (2)
    violation_handler();

  if (!p || *p <= 0)      // safety double-check
    return std::logic_arror("");
```

```
  return *p - 1;
}
```

Now, the compiler can see that if `p` is null, the control will eventually reach line marked with (2) and cause the dereference, which is UB. Since the compiler can assume that UB never happens, it can conclude that `p` is never null and can ellide the null-ness check in two places, rendering the code equivalent to:

```
int f(int * p)
{
  if (*p <= 0)            // (2)
    violation_handler();

  if (*p <= 0)            // safety double-check also compromised
    return std::logic_arror("");

  return *p - 1;
}
```

And now our program obtained undefined behavior for null pointer values of `p` only because we enabled the runtime-checking of contract annotations! The key observetion here is that the defensive check that uses logical operator AND has the short-circuiting property:

```
int f(int * p)
{
  if (!p || *p <= 0)      // null `p` never dereferenced
    return std::logic_arror("");

  return *p - 1;
}
```

Short-circuiting also occurs for a combination of `if`-statements and `return`s:

```
int f(int * p)
{
  if (!p)
    return std::logic_arror("");

  if (*p <= 0)      // null `p` never dereferenced
    return std::logic_arror("");

  return *p - 1;
}
```

It also occurs for subsequent precndition annotations, provided that the violation handler terminates:

```
int f(int * p)
  [[pre: p]]      // hopefully, the violation handler terminates
  [[pre: *p > 0]]
  ;
```

But short-circuiting is gone, when the handler allows the program flow to continue.

Avoiding this nasty effect in the language would require an introduction of a new thing into the language: *observable checkpoints* as described in [P1494r0]. Otherwise, we would have to leave it as a gothcha and teach that expressions that depend on another expressions be put in one annotation:

```
int f(int * p)
  [[pre: p && *p > 0]]
  ;
```

or

```
int f(int * p)
  [[pre: p]]
  [[pre: p && *p > 0]]
  ;
```

2. Acknowledgments

This paper summarizes the input of a number of SG21 members, offerd in the meetings and the reflector.

3. References

- [P0542r5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html).
- [CppCoreGuidelines] — Bjarne Stroustrup, Herb Sutter, "C++ Core Guidelines" (http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines).
- [GTest] — `googletest` library (https://github.com/google/googletest).
- [N3050] — David Abrahams, Rani Sharoni, Doug Gregor, "Allowing Move Constructors to Throw (Rev. 1)" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2010/n3050.html).
- [P1494r0] — S. Davis Herring, "Partial program correctness" (http://open-std.org/JTC1/SC22/WG21/docs/papers/2019/p1494r0.html).